

CIS 022

functions, intro to arrays, 1st mention of pass-by-reference

Agenda

- A useful string function
- More on parameters and return values
- Intro to arrays

A USEFUL STRING FUNCTION

- Say you've defined a string:

```
string myString = "Hello world!";
```

- You can ask the string how long it is by giving the string's name, a dot, and `length()`.
- This is a *function call* that *returns an int*. You can store that int in a variable or use it immediately (in this case, I print it out without saving it in a variable).

```
cout << myString.length() << endl;
```

- (this will be very useful indeed for the fancy printer program)

FUNCTIONS

- Unsurprisingly, we can also use functions in C++.
- The chief difference between Karel functions and C++ functions *can take input and return output*.
- We've been using the C++ main function since we started writing C++ -- every C++ program must have a main function.
- Now we're going to define new functions of our own.

First function – void function with no parameters

- Most of what's tricky about C++ functions involves *parameters* (variables you pass in to the function) and *return values* (variables your function sends out to the function that called it).
- If you've got a function that takes no parameters and returns nothing, it is conceptually very similar to Karel functions

First function – void function with no parameters

- Most of what's tricky about C++ functions involves *parameters* (variables you pass in to the function) and *return values* (variables your function sends out to the function that called it).
- If you've got a function that takes no parameters and returns nothing, it is conceptually very similar to Karel functions.
- This is a function that just prints the word "Hello!"
- When you specify a function in C++, you give its return type, its name, and then the parameters it takes.
- Its return type is *void* (this means it doesn't return anything)
- As in Karel, the parentheses after the function name are empty – it takes no parameters

```
5  
6 void annoyingFunction(){  
7     cout << "Hello!" << endl;  
8 }
```

First function – void function with no parameters

- Here's our annoying function in use:
- Notice: when you *call* a function, you give its name and then its parameters in parentheses. This function takes no parameters, so we use empty parentheses

```
1 #include <iostream>
2
3
4 using namespace std;
5
6 void annoyingFunction(){
7     cout << "Hello!" << endl;
8 }
9
10 int main(){
11     annoyingFunction();
12     annoyingFunction();
13     annoyingFunction();
14
15
16     return 0;
17 }
```

Second function – void function with one parameter

This function now takes *one* parameter – the string that we're telling it to print.

```
10
11 void generalizedAnnoyingFunction(string stringToPrint){
12     cout << stringToPrint << endl;
13
14 }
15
16 int main(){
17
18     generalizedAnnoyingFunction("Hello!");
19     string someVariable = "blahh!";
20     generalizedAnnoyingFunction(someVariable);
21
22     return 0;
23 }
```


OUR FIRST FUNCTION with parameters (other than main)

- This function is designed to raise a (positive, non-zero) number to a power.
- It takes two pieces of input...

```
5 // raises a positive number to the given power.
6 int raiseToPower(int base, int exponent){
7     for (int i = 0; i < exponent; i++){
8         result = result * base ;    // this can also be written as
9                                     // result *= base;
10
11     }
12     return result;
13 }
```



(an int named base and another int named exponent)

OUR FIRST FUNCTION with parameters (other than main)

- This function is designed to raise a (positive, non-zero) number to a power.
- It takes two pieces of input...
- ... and produces one piece of output.

```
1 #include <iostream>
2
3 using namespace std;
4
5 int raiseToPower(int base, int exponent) {
6     int result = 1;
7     for (int i = 0; i < exponent; i++){
8         result = result * base;
9     }
10    return result;
11 }
12
13 int main(){
14
15     cout << raiseToPower(2, 3) << endl;
16 }
17
18
19
```

OUR FIRST FUNCTION with parameters (other than main)

In Karel we declared a function by giving the keyword *function*, followed by the name of the function, followed by empty parentheses



```
7 function drawOneRow(){
8     while(frontIsClear()){
9         move();
10        putBeeper();
11    }
12 }
```

In C++ we declare a function by giving *the type of the return value*, followed by the name of the function, followed by *the inputs to the function* given in parentheses.

return type



parameters



```
4
5 int raiseToPower(int base, int exponent) {
6     int result = 1;
7     for (int i = 0; i < exponent; i++){
8         result = result * base;
9     }
10    return result;
11 }
12
```

(in this case, our function returns an int and takes two ints as input)

PARAMETERS

- Inputs to functions are called *parameters*. The parameters of this function are an `int` called `base` and an `int` called `exponent`.

In C++ we declare a function by giving *the type of the return value*, followed by the name of the function, followed by *the inputs to the function* given in parentheses.



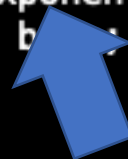
```
4
5 int raiseToPower(int base, int exponent) {
6     int result = 1;
7     for (int i = 0; i < exponent; i++){
8         result = result * base;
9     }
10    return result;
11 }
12
```

PARAMETERS

- Inputs to functions are called *parameters*. The parameters of this function are an `int` called `base` and an `int` called `exponent`.
- Parameters are variables and can be used inside the function just like any other variable.

In C++ we declare a function by giving *the type of the return value*, followed by the name of the function, followed by *the inputs to the function* given in parentheses.

```
4
5 int raiseToPower(int base, int exponent) {
6     int result = 1;
7     for (int i = 0; i < exponent; i++){
8         result = result * base;
9     }
10    return result;
11 }
12
```



here we use the exponent variable...

PARAMETERS

- Inputs to functions are called *parameters*. The parameters of this function are an `int` called `base` and an `int` called `exponent`.
- Parameters are variables and can be used inside the function just like any other variable.

In C++ we declare a function by giving *the type of the return value*, followed by the name of the function, followed by *the inputs to the function* given in parentheses.

```
4
5 int raiseToPower(int base, int exponent) {
6     int result = 1;
7     for (int i = 0; i < exponent; i++){
8         result = result * base;
9     }
10    return result;
11 }
12
```



... and here we use the base variable

RETURN VALUES and if/else

anything raised to the zeroth power is 1

- If you have if/else statements in your function, remember that all cases *must* return a variable.
- Here I've extended the previous version of the function to return the correct value when the exponent is 0

```
5 // raises a positive number to the given power.
6 int raiseToPower(int base, int exponent){
7     if (exponent == 0){
8         return 1;
9     } else {
10        int result = 1;
11        for (int i = 0; i < exponent; i++){
12            result = result * base ;    // this can also be written as
13                                       // result *= base;
14        }
15    }
16    return result;
17 }
18 }
19 }
```



RETURN VALUES

- If I left out the return statement, the compiler will complain:

compiler doesn't like this



```
5 // raises a positive number to the given power.
6 int raiseToPower(int base, int exponent){
7     if (exponent == 0){
8         cout << "Sorry, you must enter a nonzero positive number! " << endl;
9     } else {
10         int result = 1;
11         for (int i = 0; i < exponent; i++){
12             result = result * base ;    // this can also be written as
13                                         // result *= base;
14         }
15         return result;
16     }
17 }
18 }
```

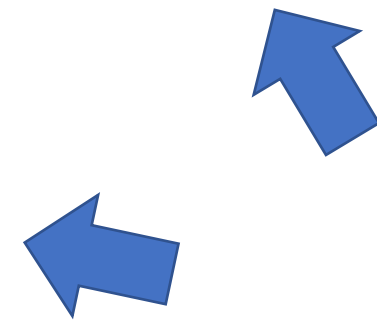
```
Bens-MacBook-Air-2:sample-code benallen$ g++ -o powerFunction powerFunction.cpp
powerFunction.cpp:18:1: warning: control may reach end of non-void function
      [-Wreturn-type]
}
^
1 warning generated.
Bens-MacBook-Air-2:sample-code benallen$
```


if you return, you exit the function

- We can actually leave out the else keyword in this function, because once we return, we leave the function and return to the function that called it.

```
5 // raises a positive number to the given power.
6 int raiseToPower(int base, int exponent){
7     if (exponent == 0){
8         return 1;
9     } else {
10         int result = 1;
11         for (int i = 0; i < exponent; i++){
12             result = result * base ;    // this can also be written as
13                                         // result *= base;
14         }
15     }
16     return result;
17 }
18 }
19 }
```

```
5 // raises a positive number to the given power.
6 int raiseToPower(int base, int exponent){
7     if (exponent == 0){
8         return 1;
9     }
10    int result = 1;
11    for (int i = 0; i < exponent; i++){
12        result = result * base ;    // this can also be written as
13                                     // result *= base;
14    }
15    return result;
16 }
17 }
```



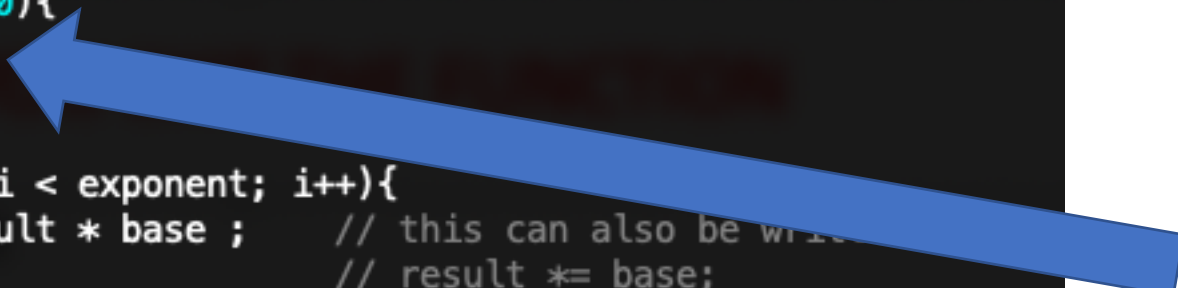
these two are
equivalent. why?

if you return, you exit the function

- We can actually leave out the else keyword in this function, because once we return, we leave the function and return to the function that called it.

```
5 // raises a positive number to the given power.
6 int raiseToPower(int base, int exponent){
7     if (exponent == 0){
8         return 1;
9     } else {
10         int result = 1;
11         for (int i = 0; i < exponent; i++){
12             result = result * base ;    // this can also be written as
13                                         // result *= base;
14         }
15     }
16     return result;
17 }
18 }
19 }
```

```
5 // raises a positive number to the given power.
6 int raiseToPower(int base, int exponent){
7     if (exponent == 0){
8         return 1;
9     }
10    int result = 1;
11    for (int i = 0; i < exponent; i++){
12        result = result * base ;    // this can also be written as
13                                     // result *= base;
14    }
15    return result;
16 }
17 }
```



because once we return a value, the function's work is done. If exponent is equal to 0, nothing after line 8 is actually executed.

our function in use

- Here's a full program that uses the raiseToPower function:

```
1 #include <iostream>
2
3 using namespace std;
4
5 // raises a positive number to the given power.
6 int raiseToPower(int base, int exponent){
7     if (exponent == 0){
8         return 1;
9     }
10    int result = 1;
11    for (int i = 0; i < exponent; i++){
12        result = result * base ;    // this can also be written as
13                                    // result *= base;
14    }
15    return result;
16 }
17
18 int main(){
19     int base;
20     int power;
21     cout << "Enter the base: " << endl;
22     cin >> base;
23     cout << "Enter the power you'll raise the base to: " << endl;
24     cin >> power;
25     int myResult = raiseToPower(base, power);
26     cout << "The result is: " << myResult << endl;
27
28     return 0;
29
30 }
```

notice how I call the
function here.

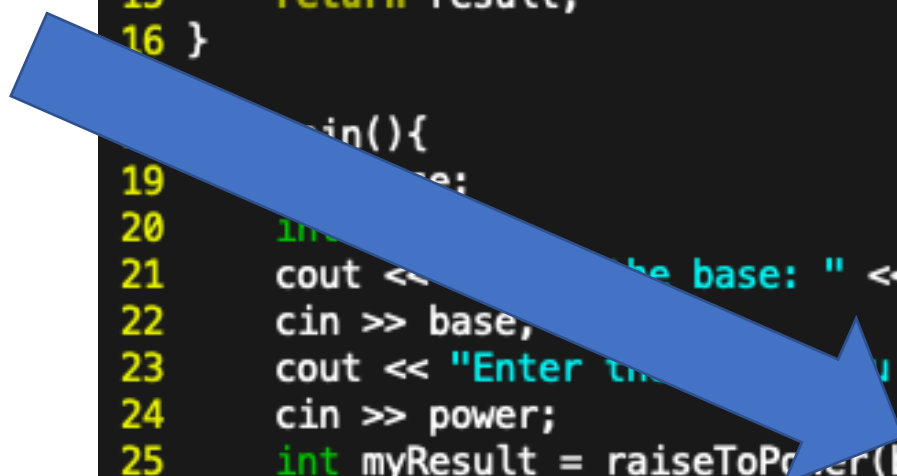


our function in use

- Here's a full program that uses the raiseToPower function:

I send in base
and power as the
arguments given
to the function

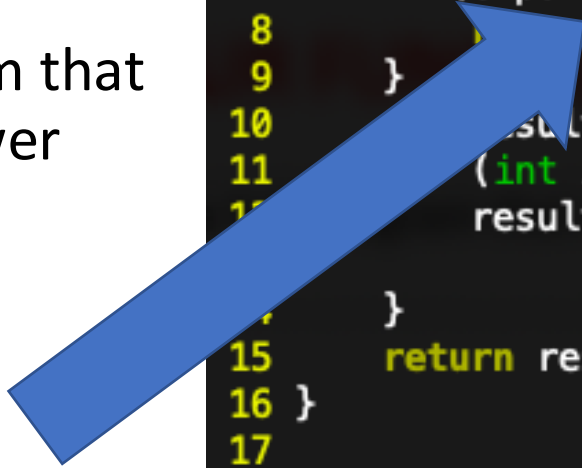
```
1 #include <iostream>
2
3 using namespace std;
4
5 // raises a positive number to the given power.
6 int raiseToPower(int base, int exponent){
7     if (exponent == 0){
8         return 1;
9     }
10    int result = 1;
11    for (int i = 0; i < exponent; i++){
12        result = result * base ;    // this can also be written as
13                                    // result *= base;
14    }
15    return result;
16 }
17
18 int main(){
19     int base;
20     int power;
21     cout << "Enter the base: " << endl;
22     cin >> base;
23     cout << "Enter the power you'll raise the base to: " << endl;
24     cin >> power;
25     int myResult = raiseToPower(base, power);
26     cout << "The result is: " << myResult << endl;
27
28     return 0;
29
30 }
```



our function in use

- Here's a full program that uses the raiseToPower function:

These *arguments* become the *parameters* when I enter raiseToPower



```
1 #include <iostream>
2
3 using namespace std;
4
5 // raises a positive number to the given power.
6 int raiseToPower(int base, int exponent){
7     if (exponent == 0){
8         return 1;
9     }
10    int result = 1;
11    for (int i = 0; i < exponent; i++){
12        result = result * base ;    // this can also be written as
                                   // result *= base;
13    }
14    return result;
15 }
16
17
18 int main(){
19     int base;
20     int power;
21     cout << "Enter the base: " << endl;
22     cin >> base;
23     cout << "Enter the power you'll raise the base to: " << endl;
24     cin >> power;
25     int myResult = raiseToPower(base, power);
26     cout << "The result is: " << myResult << endl;
27
28     return 0;
29
30 }
```

our function in use

- Here's a full program that uses the raiseToPower function:

Once raiseToPower finishes running, I store its return value in the myResult variable, and then print it on the console.

```
1 #include <iostream>
2
3 using namespace std;
4
5 // raises a positive number to the given power.
6 int raiseToPower(int base, int exponent){
7     if (exponent == 0){
8         return 1;
9     }
10    int result = 1;
11    for (int i = 0; i < exponent; i++){
12        result = result * base ;    // this can also be written as
13                                    // result *= base;
14    }
15    return result;
16 }
17
18 int main(){
19     int base;
20     int power;
21     cout << "Enter the base: " << endl;
22     cin >> base;
23     cout << "Enter the power you'll raise the base to: " << endl;
24     cin >> power;
25     int myResult = raiseToPower(base, power);
26     cout << "The result is: " << myResult << endl;
27
28     return 0;
29
30 }
```



our function in use

- Here's a full program that uses the raiseToPower function:

notice that I MUST return the value of result in order to use it in main – once we leave raiseToPower, result is *out of scope*.

```
1 #include <iostream>
2
3 using namespace std;
4
5 // raises a positive number to the given power.
6 int raiseToPower(int base, int exponent){
7     if (exponent == 0){
8         return 1;
9     }
10    int result = 1;
11    for (int i = 0; i < exponent; i++){
12        result = result * base ;    // this can also be written as
13                                    // result *= base;
14    }
15    return result;
16
17
18 int main(){
19     int base;
20     int power;
21     cout << "Enter the base: " << endl;
22     cin >> base;
23     cout << "Enter the power you'll raise the base to: " << endl;
24     cin >> power;
25     int myResult = raiseToPower(base, power);
26     cout << "The result is: " << myResult << endl;
27
28     return 0;
29
30 }
```



our function in use

I can also directly use the value returned by `raiseToPower`, instead of storing it in a variable.

```
21 cout << "Enter the base: " << endl;  
22 cin >> base;  
23 cout << "Enter the power you'll raise the base to: " << endl;  
24 cin >> power;  
25 cout << "The result is: " << raiseToPower(base, power) << endl;  
26
```



here I just print it, instead of saving it.

our function in use

BUT if I neither print nor
save the return value, it is
lost forever

```
26  
27     raiseToPower(base, power);  
28
```



this does NOTHING – we call
raiseToPower, it calculates a
result, returns that result...
but then we don't do
anything with it. It's lost
forever!

ARRAYS!

ADDING MACHINE (NOW WITH AN ARRAY)

declaring an array is a lot like declaring any other variable: give the *type* of the variable, then whatever name makes sense.

```
1  /* File: addingMachine.cpp
2  * Name: Ben Allen
3  * Class: cis6
4  * -----
5  * Short program to demonstrate arrays */
6
7  #include <iostream>
8
9  using namespace std;
10
11 int findSum(int firstArray[]){
12     int result = firstArray[0] + firstArray[1] + firstArray[2] + firstArray[3] + firstArray[4];
13     return result;
14 }
15
16 int main() {
17     int firstArray[5];
18
19     cout << "Enter five numbers and I'll tell you what they add up to: ";
20     cin >> firstArray[0] >> firstArray[1] >> firstArray[2] >> firstArray[3] >> firstArray[4];
21     cout << "Your result: "
22         << findSum(firstArray)
23         << endl;
24
25 }
```

ADDING MACHINE (NOW WITH AN ARRAY)

the difference is that you include a number in brackets after the name. This indicates *how many items* are in the array.

firstArray is an array of *five ints*.

```
1 /* File: addingMachine.cpp
2  * Name: Ben Allen
3  * Class: cis6
4  * -----
5  * Short program to demonstrate arrays */
6
7 #include <iostream>
8
9 using namespace std;
10
11 int findSum(int firstArray[]){
12     int result = firstArray[0] + firstArray[1] + firstArray[2] + firstArray[3] + firstArray[4];
13     return result;
14 }
15
16 int main() {
17     int firstArray[5];
18
19     cout << "Enter five numbers and I'll tell you what they add up to: ";
20     cin >> firstArray[0] >> firstArray[1] >> firstArray[2] >> firstArray[3] >> firstArray[4];
21     cout << "Your result: "
22         << findSum(firstArray)
23         << endl;
24 }
25 }
```



ADDING MACHINE (NOW WITH AN ARRAY)

Access an item in an array by *index*. The first index in any array is 0. The final index is the size of the array minus 1.

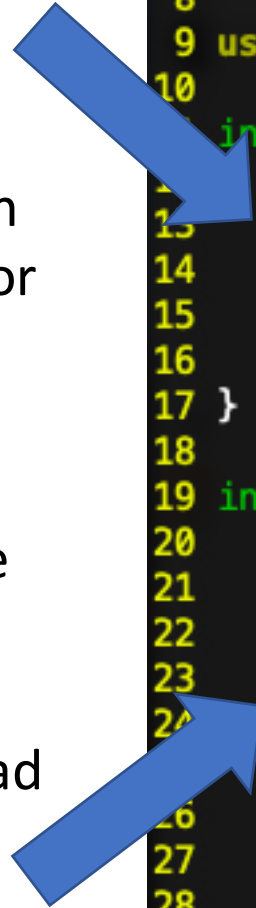
```
1 /* File: addingMachine.cpp
2  * Name: Ben Allen
3  * Class: cis6
4  * -----
5  * Short program to demonstrate arrays */
6
7 #include <iostream>
8
9 using namespace std;
10
11 int findSum(int firstArray[]){
12     int result = firstArray[0] + firstArray[1] + firstArray[2] + firstArray[3] + firstArray[4];
13     return result;
14 }
15
16 int main() {
17     int firstArray[5];
18
19     cout << "Enter five numbers and I'll tell you what they add up to: ";
20     cin >> firstArray[0] >> firstArray[1] >> firstArray[2] >> firstArray[3] >> firstArray[4];
21     cout << "Your result: "
22         << findSum(firstArray)
23         << endl;
24 }
25
```

first index

ACCESSING ARRAYS WITH FOR LOOPS

In both my main function and in findSum, I use a for loop to write to or read from the array.

The loop control variable is the *index* I want to access. I keep increasing this variable until I've read from (or written to) the entire array.

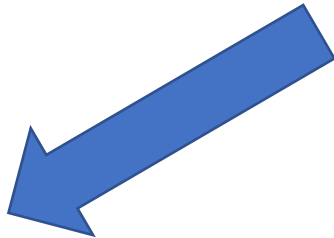


```
1  /* File: addingMachine.cpp
2  * Name: Ben Allen
3  * Class: cis6
4  * -----
5  * Short program to demonstrate arrays */
6
7  #include <iostream>
8
9  using namespace std;
10
11  int findSum(int firstArray[]){
12      int result = 0;
13      for (int index = 0; index < 5; index++){
14          result += firstArray[index];
15      }
16      return result;
17  }
18
19  int main() {
20      int firstArray[5];
21
22      cout << "Enter five numbers and I'll tell you what they add up to: ";
23      for (int index = 0; index < 5; index++){
24          cin >> firstArray[index];
25      }
26      cout << "Your result: "
27           << findSum(firstArray)
28           << endl;
29  }
30 }
```

RETURNING AN ARRAY... DOESN'T WORK

If you try to return an array, you'll get a VERY mysterious error message when you compile your program:

```
21 int incrementArray(int firstArray[]){  
22     for (int index = 0; index < SIZE; index++){  
23         firstArray[index] += 1;  
24     }  
25     return firstArray;  
26 }  
27
```



```
addingMachine.cpp:25:9: error: cannot initialize return object of type 'int' with an lvalue of type 'int *'  
    return firstArray;  
           ~~~~~  
1 error generated.  
Bens-MacBook-Air-2:arrays benallen$
```

Initializing arrays

- You can initialize an array as you declare it by giving each item, separated by commas, inside curly braces.
- When you do this, you do NOT have to give the size of the array inside brackets – you can just leave the brackets empty.

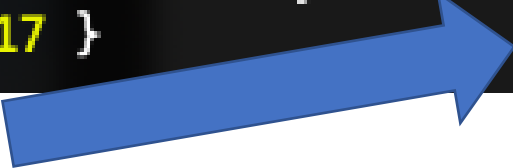
```
char helloArray[] = {'h', 'e', 'l', 'l', 'o', '!'};
```


Passing arrays into functions

function declaration:
use brackets (but no size)

- When you declare a function that takes an array as a parameter, include brackets after the name of the array.
- When you pass an array into a function as an argument, just give the array name (without brackets)

```
4
5 const int ARRAY_SIZE = 5;
6
7 void arrayPrinter(char array[]){
8     for (int index = 0; index < ARRAY_SIZE; index++){
9         cout << array[index];
10    }
11    cout << endl;
12 }
13
14 int main(){
15     char helloArray[] = {'h', 'e', 'l', 'l', 'o'};
16     arrayPrinter(helloArray);
17 }
```



passing an array to a
function: just give the
array name

Review: Accessing arrays

- Access items in arrays by giving the index of the item in brackets

```
6
7  char helloArray[] = {'h', 'e', 'l', 'l', 'o', '!'};
8
9  cout << helloArray[0] << endl;
```



```
[Bens-MacBook-Air-2:arrays benallen$ ./arrayTest
h
```

Review: Accessing every item in an array

- A common idiom is to access every item in an array using a for loop

```
6
7     char helloArray[] = {'h', 'e', 'l', 'l', 'o', '!'};
8
9     for (int index = 0; index < 6; index++){
10         cout << helloArray[index];
11     }
12     cout << endl;
```



```
[Ben's-MacBook-Air-2:arrays benallen$ ./arrayTest
hello!
```

OFF BY ONE ERRORS

- Say I try this:
- Notice that in both `initializeArray` and `printArray` I keep going until index is *more than* size.
- Is there supposed to be an item at `array[5]`?

```
7 #include <iostream>
8 using namespace std;
9
10 const int SIZE = 5;
11
12 void initializeArray(int array[]){
13     for (int index = 0; index <= SIZE; index++){
14         array[index] = index * 2;
15     }
16 }
17
18 void printArray(int array[]){
19     for (int index = 0; index <= SIZE; index++){
20         cout << array[index] << endl;
21     }
22 }
23
24 int main() {
25     int firstArray[SIZE];
26     initializeArray(firstArray);
27     printArray(firstArray);
28 }
29 }
```

OFF BY ONE ERRORS

- This yields *unpredictable results*, especially if you're dealing with 2d arrays. When you write past the end of one row, you can accidentally write to the start of the next row.

```
7 #include <iostream>
8 using namespace std;
9
10 const int SIZE = 5;
11
12 void initializeArray(int array[]){
13     for (int index = 0; index <= SIZE; index++){
14         array[index] = index * 2;
15     }
16 }
17
18 void printArray(int array[]){
19     for (int index = 0; index <= SIZE; index++){
20         cout << array[index] << endl;
21     }
22 }
23
24 int main() {
25     int firstArray[SIZE];
26     initializeArray(firstArray);
27     printArray(firstArray);
28 }
29 }
```

OFF BY ONE ERRORS

- Check out what happens when I have an off-by-one error in the printArray function, but not the initializeArray one:

this prints out
whatever happens to
be in memory
in the space past
the end of the
array



```
1 #include <iostream>
2
3 using namespace std;
4
5
6
7 #include <iostream>
8 using namespace std;
9
10 const int SIZE = 5;
11
12 void initializeArray(int array[]){
13     for (int index = 0; index < SIZE; index++){
14         array[index] = index * 2;
15     }
16 }
17
18 void printArray(int array[]){
19     for (int index = 0; index <= SIZE; index++){
20         cout << array[index] << endl;
21     }
22 }
23
24 int main() {
25     int firstArray[SIZE];
26     initializeArray(firstArray);
27     printArray(firstArray);
28 }
```

```
Bens-MacBook-Air-2:arrays benallen$ ./off-by-one
0
2
4
6
8
32766
Bens-MacBook-Air-2:arrays benallen$
```

OFF BY ONE ERRORS

- Off-by-one errors are basically the devil. Watch out for them!

this prints out
whatever happens to
be in memory
in the space past
the end of the
array



```
1 #include <iostream>
2
3 using namespace std;
4
5
6
7 #include <iostream>
8 using namespace std;
9
10 const int SIZE = 5;
11
12 void initializeArray(int array[]){
13     for (int index = 0; index < SIZE; index++){
14         array[index] = index * 2;
15     }
16 }
17
18 void printArray(int array[]){
19     for (int index = 0; index <= SIZE; index++){
20         cout << array[index] << endl;
21     }
22 }
23
24 int main() {
25     int firstArray[SIZE];
26     initializeArray(firstArray);
27     printArray(firstArray);
28 }
```

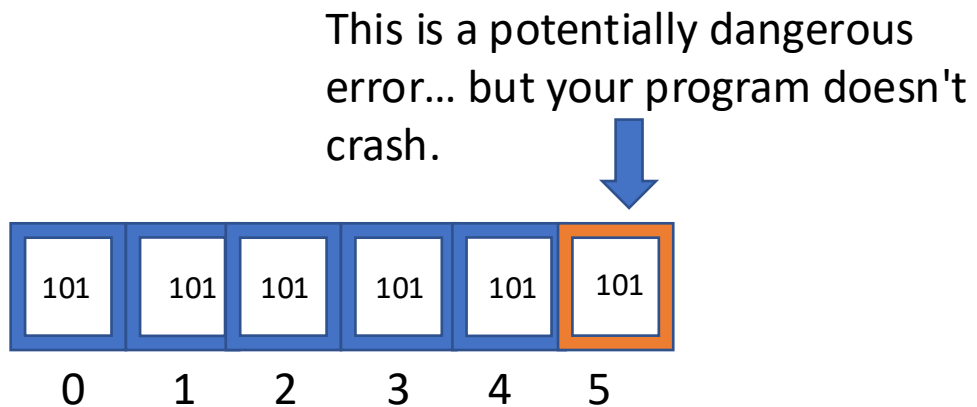
```
Bens-MacBook-Air-2:arrays benallen$ ./off-by-one
0
2
4
6
8
32766
Bens-MacBook-Air-2:arrays benallen$
```

NO BOUNDARY CHECKING IN C++

- More broadly speaking: The C++ compiler will *never* check if you're accidentally reading from or writing to a memory location that's past the end of an array.
 - This is for performance reasons: because C++ is used for relatively low-level programs, it has to be *blazingly* fast
 - Always checking if an item is off the end of an array would slow down C++ programs too much.
- Reading off an array is considered *undefined behavior*. The compiler can do literally anything the compiler writers want in response to undefined behavior

NO BOUNDARY CHECKING IN C++

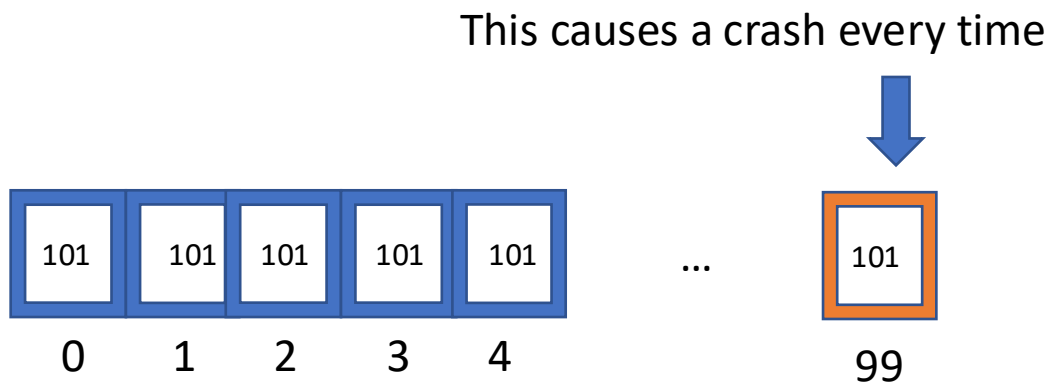
- In practice: your program will have memory access errors that *don't* cause crashes if you access an element only a few spaces off the end of the array:



```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 int main(){
7     int myArray[5];
8     for (int i = 0; i < 6; i++){
9         myArray[i] = 101;
10    }
11    return 0;
12 }
13
14
```

NO BOUNDARY CHECKING IN C++

- In practice: your program will have memory access errors that *don't* cause crashes if you access an element only a few spaces off the end of the array:



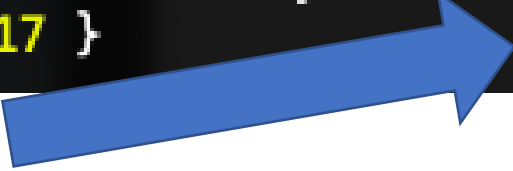
```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 int main(){
7     int myArray[5];
8     for (int i = 0; i < 100; i++){
9         myArray[i] = 101;
10    }
11    return 0;
12 }
13
```

Review: Passing arrays into functions

function declaration:
use brackets (but no size)

- When you declare a function that takes an array as a parameter, include brackets after the name of the array.
- When you pass an array into a function as an argument, just give the array name (without brackets)

```
4
5 const int ARRAY_SIZE = 5;
6
7 void arrayPrinter(char array[]){
8     for (int index = 0; index < ARRAY_SIZE; index++){
9         cout << array[index];
10    }
11    cout << endl;
12 }
13
14 int main(){
15     char helloArray[] = {'h', 'e', 'l', 'l', 'o'};
16     arrayPrinter(helloArray);
17 }
```



passing an array to a
function: just give the
array name

Common idiom

- In our previous example, we defined our array size in terms of a global constant named `ARRAY_SIZE`. If you have an array that's not defined using a global constant, it's common to write functions that take as parameters both an array and the size of that array:

```
5 void arrayPrinter(char array[], int size){
6     for (int index = 0; index < size; index++){
7         cout << array[index];
8     }
9     cout << endl;
10 }
11
12 int main(){
13     char helloArray[] = {'h', 'e', 'l', 'l', 'o'};
14     arrayPrinter(helloArray, 5);
15 }
16
```

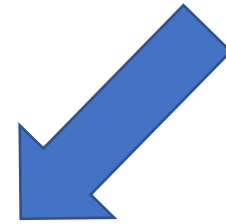
New material: changing an array in a function

- So here's something weird – and it's important:

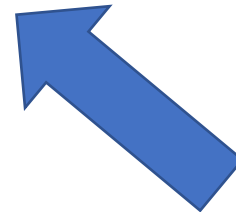
If you change an array in a function, you change it outside the function too.

(See next slide for an example)

important



important



important



Changing an array in a function

so I initialize helloArray on
line 23...



```
6
7 void arrayPrinter(char array[]){
8     for (int index = 0; index < SIZE; index++){
9         cout << array[index];
10    }
11    cout << endl;
12 }
13
14 void arrayChanger(char array[]) {
15     array[0] = 'a';
16     array[1] = 'l';
17     array[2] = 'o';
18     array[3] = 'h';
19     array[4] = 'a';
20 }
21
22 int main(){
23     char helloArray[] = {'h', 'e', 'l', 'l', 'o'};
24     arrayChanger(helloArray);
25     arrayPrinter(helloArray);
26 }
27
```

Changing an array in a function

I pass it into
arrayChanger,
which
changes each
item...



```
6
7 void arrayPrinter(char array[]){
8     for (int index = 0; index < SIZE; index++){
9         cout << array[index];
10    }
11    cout << endl;
12 }
13
14 void arrayChanger(char array[]) {
15     array[0] = 'a';
16     array[1] = 'l';
17     array[2] = 'o';
18     array[3] = 'h';
19     array[4] = 'a';
20 }
21
22 int main(){
23     char helloArray[] = {'h', 'e', 'l', 'l', 'o'};
24     arrayChanger(helloArray);
25     arrayPrinter(helloArray);
26 }
27
```

Changing an array in a function

```
6
7 void arrayPrinter(char array[]){
8     for (int index = 0; index < SIZE; index++){
9         cout << array[index];
10    }
11    cout << endl;
12 }
13
14 void arrayChanger(char array[]) {
15     array[0] = 'a';
16     array[1] = 'l';
17     array[2] = 'o';
18     array[3] = 'h';
19     array[4] = 'a';
20 }
21
22 int main(){
23     char helloArray[] = {'h', 'e', 'l', 'l', 'o'};
24     arrayChanger(helloArray);
25     arrayPrinter(helloArray);
26 }
27
```

... and when I print it out...



Changing an array in a function

```
6
7 void arrayPrinter(char array[]){
8     for (int index = 0; index < SIZE; index++){
9         cout << array[index];
10    }
11    cout << endl;
12 }
13
14 void arrayChanger(char array[]) {
15     array[0] = 'a';
16     array[1] = 'l';
17     array[2] = 'o';
18     array[3] = 'h';
19     array[4] = 'a';
20 }
21
22 int main(){
23     char helloArray[] = {'h', 'e', 'l', 'l', 'o'};
24     arrayChanger(helloArray);
25     arrayPrinter(helloArray);
26 }
27
```

... and when I print it
out... it's changed?



```
Bens-MacBook-Air-2:arrays benallen$ ./arrayTest
aloha
```

WHAT.

```
6
7 void arrayPrinter(char array[]){
8     for (int index = 0; index < SIZE; index++){
9         cout << array[index];
10    }
11    cout << endl;
12 }
13
14 void arrayChanger(char array[]) {
15     array[0] = 'a';
16     array[1] = 'l';
17     array[2] = 'o';
18     array[3] = 'h';
19     array[4] = 'a';
20 }
21
22 int main(){
23     char helloArray[] = {'h', 'e', 'l', 'l', 'o'};
24     arrayChanger(helloArray);
25     arrayPrinter(helloArray);
26 }
27
```

WHAT???

... and when I print it
out... it's changed?.



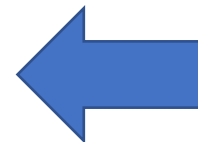
```
Bens-MacBook-Air-2:arrays benallen$ ./arrayTest
aloha
```

THAT'S NOT HOW IT WORKS FOR OTHER THINGS

- We know that when we pass an int or a char or a string into a function, C++ makes a *copy* of whatever's in that int or char or string.
- When we change the copy, the original remains unchanged

```
23 void stringChanger(string myString){
24     myString = "aloha";
25
26 }
27
28 int main(){
29     char helloArray[] = {'h', 'e', 'l', 'l', 'o'};
30     string helloString = "hello";
31     stringChanger(helloString);
32     cout << helloString << endl;
33 }
```

```
Bens-MacBook-Air-2:arrays benallen$ ./arrayTest
hello
```

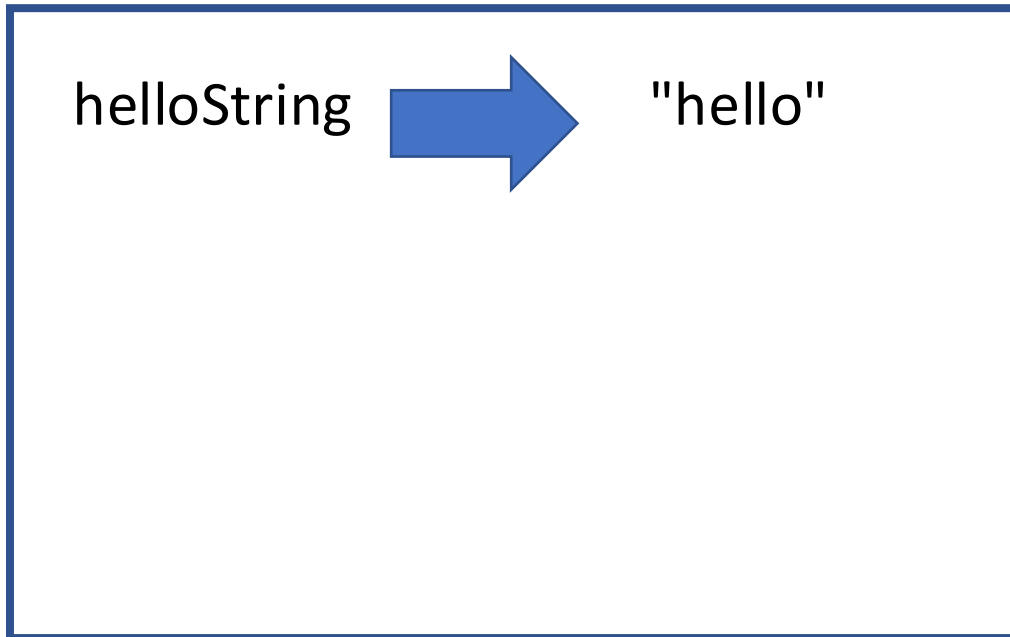


although we change the copy of the string in stringChanger, the original version back in main doesn't change

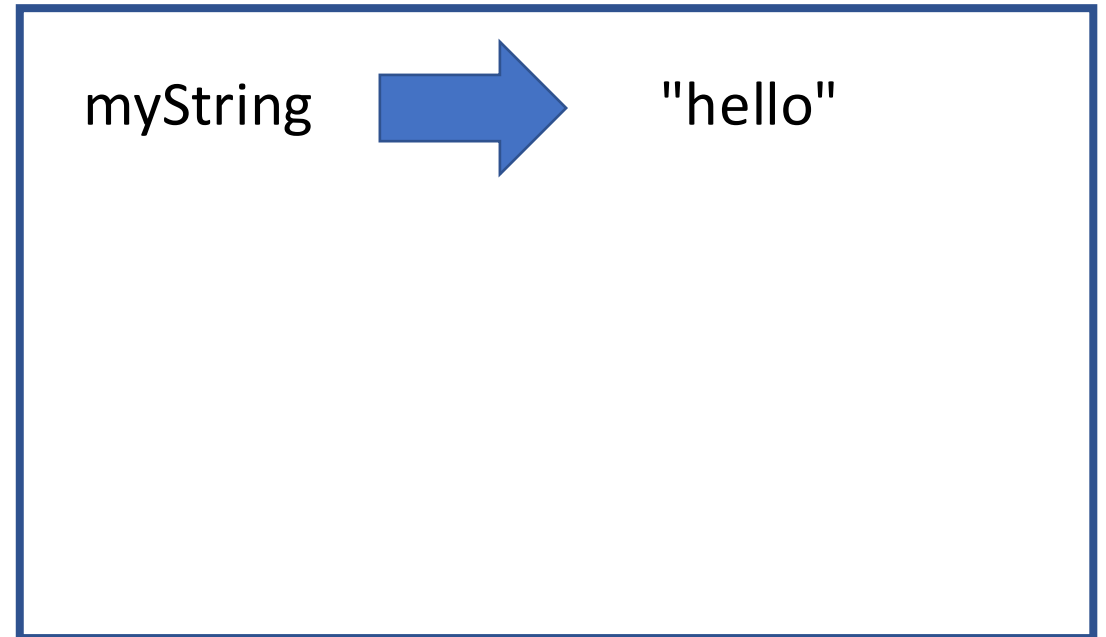
How it looks in memory (strings and ints and chars and doubles and floats...)

- myString in stringChanger and helloString in main point to different places in memory...

main:



stringChanger



How it looks in memory (strings and ints and chars and doubles and floats...)

- ... so when we change the string in stringChanger, it's left unchanged in main

main:

helloString → "hello"

stringChanger

myString → "aloha"

How it looks in memory (arrays)

- When we pass an array, C++ *doesn't* make a copy. Instead, the name of the array in the function we call points *to exactly the same space in memory* as the name of the array in the original function.

main:

helloArray[]

{'h', 'e', 'l', 'l', 'o'}

arrayChanger:

helloArray[]

```
6
7 void arrayPrinter(char array[]){
8     for (int index = 0; index < SIZE; index++){
9         cout << array[index];
10    }
11    cout << endl;
12 }
13
14 void arrayChanger(char array[]) {
15     array[0] = 'a';
16     array[1] = 'l';
17     array[2] = 'o';
18     array[3] = 'h';
19     array[4] = 'a';
20 }
21
22 int main(){
23     char helloArray[] = {'h', 'e', 'l', 'l', 'o'};
24     arrayChanger(helloArray);
25     arrayPrinter(helloArray);
26 }
```

How it looks in memory (arrays)

- So when we change it in arrayChanger, we change it back in main as well

main:

helloArray[]

{ 'a', 'l', 'o', 'h', 'a' }

arrayChanger:

helloArray[]

```
6
7 void arrayPrinter(char array[]){
8     for (int index = 0; index < SIZE; index++){
9         cout << array[index];
10    }
11    cout << endl;
12 }
13
14 void arrayChanger(char array[]) {
15     array[0] = 'a';
16     array[1] = 'l';
17     array[2] = 'o';
18     array[3] = 'h';
19     array[4] = 'a';
20 }
21
22 int main(){
23     char helloArray[] = {'h', 'e', 'l', 'l', 'o'};
24     arrayChanger(helloArray);
25     arrayPrinter(helloArray);
26 }
```

How it looks in memory (arrays)

- So when we change it in arrayChanger, we change it back in main as well

main:

helloArray[]

{'a', 'l', 'o', 'h', 'a'}

arrayChanger:

helloArray[]

you might be thinking...



Here's why:

- Arrays are pass-by-reference because arrays might be *very large*. If we make an extra copy of a large array every time we call a function, it might significantly slow down our programs. Therefore: arrays are pass-by-reference.

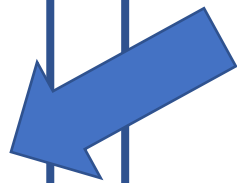
main:

helloArray[]

{ 'a', 'l', 'o', 'h', 'a' }

arrayChanger:

helloArray[]



Here's why:

- Arrays are pass-by-reference because arrays might be *very large*. If we make an extra copy of a large array every time we call a function, it might significantly slow down our programs. Therefore: arrays are pass-by-reference.

main:

helloArray[]

{'a', 'l', 'o', 'h', 'a'}

arrayChanger:

helloArray[]

ok.



TERMINOLOGY

- Here's the terminology we use to explain the difference between things that are passed like ints and strings, and things that are passed by arrays:
 - strings, ints, chars, and other simple variables are *pass by value*
 - arrays are *pass by reference*
- Learn these terms well! We'll be using them a lot.